

# LLD Practice Platform — Design Note

System Architecture, MVP Specification, Classes, User Flow & Trade-Offs

Author: Satvik Sharma | Project: LLD Practice Platform | Status: Completed MVP

## 1. Explanation of the MVP (Minimum Viable Product)

The **LLD Practice Platform** addresses a widespread industry problem: software engineers preparing for Low-Level Design (LLD) and Object-Oriented Design (OOD) interviews lack an interactive workbench providing objective, standardized, and structured feedback. Unlike competitive programming, design problems do not compile into a simple binary pass/fail result. Instead, design quality rests upon class responsibilities, encapsulation, extensibility, abstraction, and edge-case handling.

### Core MVP Objectives:

- **Problem Discovery:** Present a curated library of classical OOD problems with varied difficulties (Easy to Hard).
- **Guided Design Formulation:** Provide a 7-section structured input schema that trains candidates to think methodically: *Requirements* → *Assumptions* → *Classes* → *Responsibilities* → *Relationships* → *Design Decisions* → *Edge Cases*.
- **Dual Evaluation Engine:** Offer both a deterministic *Rule-Based Evaluator* (for structural checks) and an *AI Evaluator* (for deeper heuristic design reasoning).
- **Diagnostic Rubric:** Score solutions across 8 standardized software design criteria (1-10 each) with a normalized total score (0-100).
- **History & Iteration:** Allow engineers to track past attempts and measure design improvements over time.

**MVP Scope Boundary:** The MVP delivers full end-to-end functionality (problem browsing, session state management, attempt creation, dual evaluation execution, result visualization, and attempt history) with zero third-party database dependency, enabling immediate local execution and lightweight portability.

## 2. User Flow & State Progression

The user experience is designed to mirror a real-world technical design interview while maintaining absolute state integrity.

```
[ Problems Catalog ] | Select problem & click 'Start Practice' v [ Practice Workbench ] <-- POST /api/practice/problems/:id/attempts (State: DRAFT) | Fill 7-field design formulation | Select Engine: [ Rule-Based Evaluator ] or [ AI Evaluator ] | Click 'Submit Design' v [ Evaluation Pipeline ] <-- POST /api/practice/attempts/:id/submission | Transition: DRAFT -> SUBMITTED -> EVALUATING -> COMPLETED v [ Results Dashboard ] <-- Shows Overall Score /100 & 8 Detailed Feedback Cards |-- Click 'Try Again' --> Returns to new Practice attempt +-- Click 'History' --> Displays previous scores & completion dates
```

### Step-by-Step Flow Description:

- Discovery (ProblemsPage):** User browses 4 challenges (*Parking Lot*, *Vending Machine*, *Elevator System*, *Library Management*) with difficulty tags and requirement previews.
- Session Initialization (PracticePage):** Opening a problem issues a POST request creating a unique Attempt entity initialized to DRAFT status.
- Design Formulation:** The candidate inputs details into 7 focused text areas, guided by contextual tips.
- Engine Selection:** The user toggles between the fast **Rule-Based Evaluator** or the **AI Evaluator**.
- Submission & Processing:** Submitting locks the attempt into SUBMITTED, moves to EVALUATING, runs the evaluator, and transitions to COMPLETED.
- Feedback Review (ResultPage):** The candidate receives an overall score (/100) and 8 criterion cards with scores (/10),

evidence, concerns, suggestions, and confidence levels.

**7. Longitudinal Tracking (HistoryPage):** Candidates review past scores and compare previous submissions to evaluate progress.

### 3. Classes & Object-Oriented Architecture

The codebase follows Domain-Driven Design (DDD) with clear separation of Domain Entities, Evaluator Strategies, and Service Layers:

| Class / Module                                            | Layer      | Key Attributes & Methods                                                                  | Architectural Role                                                  |
|-----------------------------------------------------------|------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Problem                                                   | Domain     | id, title, description, difficulty, requirements                                          | Represents the challenge specifications and constraints.            |
| Attempt                                                   | Domain     | id, problemId, status, submit(), evaluate(), complete()                                   | Encapsulates the state machine lifecycle (DRAFT → COMPLETED).       |
| Submission (Abstract)<br>TextSubmission                   | Domain     | requirements, assumptions, classes, responsibilities, relationships, decisions, edgeCases | Encapsulates candidate solution data. Extensible for diagrams/code. |
| Evaluation                                                | Domain     | evaluatorType, overallScore, feedbackItems, calculateOverallScore()                       | Aggregates 8 criteria feedback items and computes score.            |
| Feedback                                                  | Domain     | criterionId, score, evidence, concern, suggestion, confidence                             | Atomic feedback unit for a single rubric dimension.                 |
| Evaluator (Abstract)<br>RuleBasedEvaluator<br>AIEvaluator | Evaluators | getType(), evaluate({ attemptId, submission })                                            | Strategy Pattern: concrete evaluators can be plugged in seamlessly. |
| ProblemService                                            | Service    | addProblem(), getProblem(), getAllProblems()                                              | Problem registry and catalog retrieval.                             |
| PracticeService                                           | Service    | startAttempt(), submitAttempt(), completeAttempt(), getAttempt()                          | Orchestrates attempt lifecycle; relies on injected ProblemService.  |
| EvaluationService                                         | Service    | registerEvaluator(), evaluate(type, payload), getEvaluation()                             | Evaluator registry and execution dispatcher.                        |

### 4. Key Design Decisions & Architectural Trade-offs

Every production system balances technical purity against practical execution constraints. The key trade-offs made in this architecture include:

| Architectural Decision  | Option Chosen in MVP                            | Alternative Considered                  | Trade-off Justification                                                                                                                                                                                                |
|-------------------------|-------------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Persistence Layer       | Thread-safe In-Memory Storage (JavaScript Maps) | PostgreSQL / MongoDB database           | <b>Benefit:</b> Zero-friction setup; instant unit test execution without database seeding.<br><b>Trade-off:</b> State does not persist across server restarts (ideal for MVP, can swap with Repository pattern later). |
| Evaluation Architecture | Pluggable Strategy Pattern (Rule-Based + AI)    | Direct External LLM API dependency only | <b>Benefit:</b> Works 100% offline, zero API key dependency, deterministic speed (0ms response).<br><b>Trade-off:</b> Rule-based evaluator is heuristic; AI evaluator provides more nuanced insights.                  |

|                             |                                       |                                                |                                                                                                                                                                                                            |
|-----------------------------|---------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Execution Model</b>      | Synchronous In-Process Execution      | Asynchronous Message Queue (RabbitMQ / BullMQ) | <b>Benefit:</b> Minimal operational complexity; client receives immediate feedback in a single HTTP request.<br><b>Trade-off:</b> Long-running multi-agent evaluators will require worker queues at scale. |
| <b>Input Structure</b>      | 7 Structured Domain Fields            | Single Freeform Markdown Box                   | <b>Benefit:</b> Forces candidate to organize thoughts into LLD principles (responsibilities, relationships, patterns).<br><b>Trade-off:</b> Slightly more rigid than a blank text document.                |
| <b>Component Decoupling</b> | Dependency Injection (DI) in Services | Global Singleton Instances                     | <b>Benefit:</b> Enables isolated unit testing, easy mocking of ProblemService or evaluators.<br><b>Trade-off:</b> Requires explicit wiring in server bootloader.                                           |

---

**Document Sign-off:**

This Design Note documents the complete technical specification, design patterns, and operational trade-offs implemented in the **LLD Practice Platform**. The platform successfully bridges the pedagogical gap in software engineering interviews through rigorous object-oriented architecture and deterministic evaluation.

**Prepared by:** Satvik Sharma | **Engineering Lead & Platform Author**